



BUENAS PRÁCTICAS DE PROGRAMACIÓN

Unidad 3

Integrantes

1. Bruno Mazzi
2. Fabián Trangoni

GUÍA DE
ACTIVIDADES
PRÁCTICAS
“Desafío de
mejora de
código”

METODOLOGÍA
DE
SISTEMAS II
Septiembre de
2026

Contenido

Aspectos Generales	2
Propósito	2
Objetivos	2
Organización	3
ACTIVIDAD 1 — Diagnóstico del código	3
ACTIVIDAD 2 — Mejoramos el código	4
ACTIVIDAD 3 — Refactorización y documentación	4
ACTIVIDAD 4 — Registrar el trabajo	5
ACTIVIDAD 5 — Code Review	6
ACTIVIDAD 6 — Cierre y puesta en común	6
Entrega Actividad	6
Co-Evaluación	7

"Desafío de mejora de código"



"Tienen 3 horas para tomar el código que ya desarrollaron y transformarlo en una versión de mayor calidad aplicando buenas prácticas."

Aspectos Generales

Aplicación de buenas prácticas sobre código existente

Duración: 3 horas

Modalidad: Práctica

Trabajo: En equipo

Recurso: Código fuente proporcionado previamente por los equipos de trabajo

Herramienta: Plataformas utilizadas por los equipos

Propósito

Aplicar buenas prácticas de programación sobre un código existente, identificando problemas de calidad y realizando mejoras que permitan obtener un código **más legible, mantenible, organizado y documentado**, sin modificar su funcionalidad.

El documento plantea que un programa no debe evaluarse solamente por funcionar correctamente, sino también por la facilidad con que puede ser comprendido, modificado, probado y ampliado.

Objetivos

Al finalizar la actividad, el estudiante será capaz de:

- identificar problemas de calidad en un código existente;
- aplicar principios básicos de **Clean Code**;
- utilizar convenciones de codificación;
- mejorar la documentación;
- realizar una refactorización simple;
- utilizar commits descriptivos;
- crear una rama de trabajo;
- realizar una revisión básica del código de un compañero.

Organización



Etapa	Actividad	Tiempo
1	Diagnóstico del código	30 min
2	Aplicación de Clean Code	40 min
3	Refactorización y documentación	35 min
4	Registrar el trabajo	25 min
5	Code Review entre pares	20 min
6	Puesta en común	10 min
7	Coevaluación	20 min
Total		180 min

ACTIVIDAD 1 — Diagnóstico del código



Consigna

Esta actividad será realizada por el equipo evaluador. Trabaje sobre el código proporcionado por el equipo asignado.

Durante esta primera etapa NO modifique el código.

Analice el programa e identifique al menos **5 problemas o aspectos que podrían mejorarse**.

Buscar específicamente:

- nombres poco claros;
- funciones demasiado extensas;
- código repetido;
- comentarios innecesarios o insuficientes;
- problemas de indentación o formato;
- responsabilidades mezcladas;
- estructuras condicionales complejas;
- falta de modularización.

Estos aspectos se corresponden con los principios de Clean Code desarrollados en el documento.

Completar

Nº	Problema	Ubicación	¿Por qué debería mejorarse?
1	Hay nombres de variables de una sola letra que no explican qué guardan.	«q» en NoticiasAdminPage.jsx (línea 55), InscripcionesAdminPage.jsx (55), MensajesAdminPage.jsx (31),	Para saber qué es «q» hay que leer las líneas siguientes. Peor todavía: la misma letra significa cosas distintas según el archivo, porque «h» es un manejador de teclado en Navbar y la hora actual en los dashboards. El

Nº	Problema	Ubicación	¿Por qué debería mejorarse?
		OpinionesAdminPage.jsx (35) y UsuariosAdminPage.jsx (44); «h» en Navbar.jsx (153) y en los tres dashboards; «m» en InscripcionPage.jsx (30).	nombre debería decir qué contiene sin que haga falta investigar.
2	Componente con demasiadas responsabilidades y función excesivamente extensa.	src/components/layout/Navbar.jsx (459 líneas; el componente Navbar ocupa ~270 y concentra 6 useEffect)	Un mismo componente resuelve el catálogo de navegación, el listener de scroll, los dropdowns de escritorio, el menú mobile en portal, el menú de usuario y el cierre de sesión. Incumple el principio de Responsabilidad Única (SRP): tiene seis motivos distintos para cambiar, y modificar el menú mobile obliga a leer toda la barra.
3	El proyecto tiene una carpeta de estilos y una paleta definida, pero los estilos igual se escriben dentro de los componentes.	68 estilos en línea (style={{...}}) repartidos en 20 archivos, y los colores #E8612C y #1B3A6B escritos a mano 24 veces en 13 archivos, aunque tailwind.config.js ya define brand-azul y brand-naranja.	Conviven tres formas distintas de dar estilo a lo mismo: globals.css, las clases de Tailwind y el estilo escrito en el componente. Si mañana cambia un color institucional hay que salir a buscarlo archivo por archivo, y seguro queda alguno viejo. Teniendo la paleta ya definida en un solo lugar, lo lógico es usarla siempre desde ahí.
4	Valores mágicos: colores y medidas escritos a mano en lugar de los tokens del tema.	24 apariciones de #E8612C y #1B3A6B en 13 archivos .jsx (Navbar, PageHero, HeroSection, NivelesPage, LoginPage, etc.), existiendo brand-azul y brand-naranja en tailwind.config.js.	La identidad visual queda duplicada fuera de su única fuente de verdad. Un cambio de color institucional obliga a buscar y reemplazar en trece archivos, con riesgo cierto de dejar inconsistencias. Lo mismo ocurre con medidas repetidas como top-[68px] y z-[100].
5	Documentación desactualizada y convenciones de codificación no verificables.	README.md (estructura del proyecto y guía de ejecución) y package.json (script «lint»).	El README describe archivos que no existen (src/lib/validations.js, src/styles/animations.css, docs/ROLES_Y_PERMISOS.md), ubica las páginas en src/pages/ cuando hoy están en public/, admin/ y auth/, indica una URL de clonado que no corresponde a este repositorio y menciona deploy en Vercel siendo Netlify. Además el script «lint» no puede ejecutarse porque no hay .eslintrc ni eslint.config.js. Una documentación incorrecta desorienta más que la ausencia de documentación.

Pregunta

¿El programa funciona correctamente? ¿Significa esto que necesariamente está bien desarrollado? Explique.

Respuesta. Sí, el programa funciona: compila, está desplegado en Netlify y todas las funcionalidades responden. Pero funcionar solo acredita la corrección externa del software, es decir, que hoy hace lo que el usuario espera. No dice nada sobre su calidad interna, que es lo que determina cuánto cuesta modificarlo mañana. En este proyecto la duplicación de los hooks y del CRUD de administración, los colores repetidos en trece archivos o el README que describe una estructura inexistente no producen ningún error visible, y sin embargo encarecen cada cambio futuro y aumentan la probabilidad de introducir defectos al tocar una copia y olvidar las otras. Eso es deuda técnica: código que funciona hoy y es caro mañana. Por lo tanto, funcionar es condición necesaria pero no suficiente para afirmar que un programa está bien desarrollado.

ACTIVIDAD 2 — Mejoramos el código



Devuelva el código al equipo para que, Ahora sí, modifiquen el código.

Aplique **al menos cinco mejoras**, incluyendo obligatoriamente:

A. Nombres significativos

Cambie variables, funciones o métodos cuyos nombres no permitan comprender claramente su propósito.

El documento propone utilizar nombres que expresen claramente la intención del elemento.

B. Funciones con responsabilidad definida

Identifique al menos una función que realice demasiadas tareas y divídala cuando sea posible.

C. Eliminación de duplicación

Busque código repetido y determine si puede convertirse en una función o componente reutilizable.

D. Legibilidad

Mejore:

- indentación;
 - espacios;
 - organización;
 - estructuras condicionales;
 - formato general.
-

E. Comentarios

Elimine comentarios que simplemente expliquen lo que el código ya expresa claramente y agregue documentación cuando sea necesario explicar una decisión o una situación no evidente.

Registro de modificaciones

Mejora	Antes	Después	Justificación
Nombre de variable	let q = supabase.from('noticias') .select(...), repetido en cinco páginas del panel de administración.	let consultaNoticias = supabase.from('noticias') .select(...)	«q» no dice nada y obliga a seguir leyendo para entender qué guarda. Con el nombre completo se entiende de una. Se corrigen también «h» (que en un archivo era la tecla presionada y en otro la hora del día) y «m» (la diferencia de meses al calcular la edad).
Nombre de función	const input = (err) => `w-full border rounded-xl ...` en EmpleosAdminPage.jsx, y el mismo helper llamado inputClass(hasError) en InscripcionPage.jsx.	export function claseCampo(tieneError) en src/utls/estilosFormulari o.js, importada por ambas páginas.	«input» nombra un elemento del DOM, no lo que la función hace (devolver clases CSS). Además el mismo concepto tenía dos nombres distintos en dos archivos. Un único nombre para un único concepto, y una sola definición para las once pantallas que hoy repiten esa cadena de clases.
Función extensa	Navbar(): un único componente de ~270 líneas con 6 useEffect, el catálogo NAV_ITEMS, los dropdowns de escritorio, el menú mobile en portal y el menú de usuario.	Navbar() como composición de <NavbarEscritorio >, <MenuUsuario >, <MenuMobile >, con los efectos de interfaz agrupados en el hook useNavbarUI() y NAV_ITEMS movido a src/types/navegacion.js.	Cada pieza pasa a tener una sola razón para cambiar (SRP). Se puede leer, probar y modificar el menú mobile sin recorrer la barra completa, y el catálogo de secciones deja de estar mezclado con la lógica de presentación.
Código duplicado	useGaleria, useOpiniones, useInscripciones y useNoticias repiten el flag «cancelado», setCargando(true), el try/catch/finally y el	Hook genérico useSupabaseQuery(const ruirQuery, dependencias) que devuelve { datos, cargando, error }; cada hook queda en 5 a 10	Aplica DRY: la carga, la cancelación al desmontar y el tratamiento de errores quedan en un único lugar, se corrigen una sola vez y se prueban una sola vez. Los hooks pasan a expresar

Mejora	Antes	Después	Justificación
	armado del mensaje de error (~35 líneas cada uno, cerca de 100 líneas equivalentes).	líneas y solo declara su consulta.	únicamente qué datos piden, no cómo se piden.
Comentario	/* Componente de campo de formulario */ sobre function Campo(...) en InscripcionPage.jsx: repite lo que el código ya dice. En cambio RANGOS_NIVEL.inicial = { min: 2, max: 5, rango: '3 a 5 años' } no explica por qué el mínimo (2) no coincide con la etiqueta (3 años).	Se elimina el comentario redundante y se documenta la decisión: «min 2: se admite al aspirante que cumple 3 años antes del inicio del ciclo lectivo». Si la diferencia no fuera intencional, corresponde corregir el valor a min: 3.	El comentario valioso explica el porqué de una decisión no evidente; el que traduce el código a castellano solo agrega ruido y queda desactualizado al primer cambio. La regla de edad es exactamente el caso que sí necesita comentario, porque no se deduce leyendo.

Condición

El programa debe continuar funcionando exactamente igual que antes de las modificaciones.

ACTIVIDAD 3 — Refactorización y documentación



Seleccione **dos problemas** detectados durante el diagnóstico y realice una refactorización.

El documento establece que refactorizar consiste en modificar la estructura interna del código **sin alterar su comportamiento externo**. Entre los ejemplos se encuentran dividir funciones, mejorar nombres, eliminar duplicación, simplificar condicionales y extraer componentes.

Ejemplo de elección

Problema 1: función demasiado extensa

→ dividir en dos o más funciones.

Problema 2: código repetido

→ crear una función reutilizable.

Refactorización 1 — Eliminar duplicación (problema 3 del diagnóstico)

Antes. Cuatro hooks con la misma estructura repetida, unas 35 líneas cada uno, donde lo único distinto es la consulta:

```
useEffect(() => {
  let cancelado = false
  async function fetchImagenes() {
    setCargando(true); setError(null)
    try {
      let query = supabase.from('galeria').select(...).eq('activa', true)
```

```

    const { data, error: supabaseError } = await query
    if (supabaseError) throw supabaseError
    if (!cancelado) setImagenes(data ?? [])
  } catch (err) {
    if (!cancelado) { console.error(...); setError('No pudimos cargar las imágenes.') }
  }
  } finally { if (!cancelado) setCargando(false) }
}
fetchImagenes()
return () => { cancelado = true }
}, [categoriaId])

```

Después. Un único hook en `src/hooks/useSupabaseQuery.js` concentra carga, cancelación y errores:

```

export function useSupabaseQuery(construirQuery, dependencias = [], mensajeError) {
  const [datos, setDatos] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    let cancelado = false
    setCargando(true); setError(null)
    construirQuery()
      .then(({ data, error: errorConsulta }) => {
        if (cancelado) return
        if (errorConsulta) throw errorConsulta
        setDatos(data ?? [])
      })
      .catch(() => { if (!cancelado) setError(mensajeError) })
      .finally(() => { if (!cancelado) setCargando(false) })
    return () => { cancelado = true }
  }, dependencias)

  return { datos, cargando, error }
}

// useGaleria.js queda reducido a declarar su consulta:
export function useGaleria(categoriaId = null) {
  const { datos: imagenes, cargando, error } = useSupabaseQuery(
    () => {
      const q = supabase.from('galeria').select(CAMPOS).eq('activa', true)
      return categoriaId ? q.eq('categoria_id', categoriaId) : q
    },
    [categoriaId],
    'No pudimos cargar las imágenes.',
  )
  return { imagenes, cargando, error }
}

```

Comportamiento externo. Idéntico: los componentes siguen recibiendo { imagenes, cargando, error } con los mismos valores y en el mismo orden. Lo único que cambia es dónde vive la lógica compartida. Se eliminan alrededor de cien líneas duplicadas.

Refactorización 2 — Dividir una función extensa (problema 2 del diagnóstico)

Antes. `Navbar.jsx` tiene 459 líneas. El componente `Navbar` concentra seis `useEffect` (scroll, cierre por navegación, tecla Escape, bloqueo del scroll del body, clic fuera del menú de usuario), el catálogo `NAV_ITEMS`, la barra de escritorio, el menú de usuario y el menú mobile renderizado en un portal.

Después. Se conserva el mismo árbol de elementos, redistribuido:

src/types/navegacion.js	→ NAV_ITEMS (datos, sin JSX)
src/components/layout/useNavbarUI.js	→ los 6 useEffect y el estado de apertura
src/components/layout/NavbarEscritorio.jsx	→ barra + NavDropdown
src/components/layout/MenuUsuario.jsx	→ avatar, panel y cierre de sesión
src/components/layout/MenuMobile.jsx	→ portal + MobileNavItem
src/components/layout/Navbar.jsx	→ composición (~40 líneas)

Comportamiento externo. Idéntico: mismas rutas, mismas animaciones, mismo comportamiento del menú al navegar con el botón atrás del navegador. Cambia únicamente la organización interna. Cada archivo pasa a tener una sola razón para cambiar, que es lo que exige el principio de Responsabilidad Única.

Documentación

Agregue documentación mínima al proyecto:

README.md

Debe contener:

1. Nombre del proyecto.
2. Descripción breve.
3. Cómo ejecutar el programa.
4. Funcionalidades principales.
5. Integrantes.

La documentación del proyecto mediante un README es uno de los niveles de documentación mencionados en el material de estudio.

Estado del README y correcciones aplicadas

El proyecto ya tenía README.md, pero desactualizado respecto del código. Se reescribió verificando cada afirmación contra el repositorio:

- Nombre del proyecto: Educar para Transformar — Sistema Institucional (correcto, se conserva).
 - Descripción breve: sitio institucional del Centro Educativo con área pública, autenticación por roles y panel de administración de contenido.
 - Cómo ejecutar: se corrigió la URL de clonado, que apuntaba a un repositorio inexistente (educar-para-transformar en lugar de Proyecto-Sistema-Universitario), y la mención a Vercel, siendo que el deploy es en Netlify según netlify.toml.
 - Funcionalidades principales: se enumeran las rutas reales (home, quiénes somos, niveles, bienestar, noticias con sección interna por rol, galería, inscripción, empleo, contacto, login/registro, dashboard por rol y siete pantallas de administración).
 - Integrantes: Gonzalo Cerqueiro y Lautaro Fernández, con el reparto por Historia de Usuario (ya figuraba y se mantiene).
 - Estructura del proyecto: se reemplazó el árbol de carpetas, que listaba archivos inexistentes (src/lib/validations.js, src/styles/animations.css, docs/ROLES_Y_PERMISOS.md) y ubicaba las páginas en src/pages/ cuando están en public/, admin/ y auth/.
-

Según el material, el README es el nivel de documentación externa del proyecto: el que permite que una persona ajena ponga el sistema en marcha. Por eso una estructura de carpetas equivocada no es un detalle cosmético, sino documentación que induce al error.

ACTIVIDAD 4 — Registrar el trabajo

🕒 **30 minutos**

Esta etapa permite aplicar el concepto de control de versiones sobre el código que los estudiantes ya poseen.

Consigna

- Crear una rama para las modificaciones:
- mejora-buenas-practicas
- Realizar al menos **3 commits**.
- Por ejemplo:
- feat: mejorar estructura del programa
- refactor: mejorar nombres y funciones
- docs: agregar README

Verificar y mostrar al docente.

El estudiante deberá poder mostrar:

- rama creada;
- commits realizados;
- archivos modificados;
- historial del proyecto.

Comandos ejecutados

```
git checkout -b mejora-buenas-practicas

git add src/hooks/useSupabaseQuery.js src/hooks/useGaleria.js \
      src/hooks/useOpiniones.js src/hooks/useInscripciones.js
git commit -m "refactor: extraer useSupabaseQuery y eliminar la duplicación de los
hooks de datos"

git add src/components/layout/ src/types/navegacion.js src/utils/estilosFormulario.js
git commit -m "refactor: dividir Navbar en subcomponentes y unificar nombres de
variables"

git add README.md
git commit -m "docs: actualizar el README con la estructura real del proyecto"

git push -u origin mejora-buenas-practicas
```

Verificación para mostrar al docente

```
git branch          # rama mejora-buenas-practicas creada y activa
git log --oneline -3 # los tres commits, en formato Conventional Commits
git show --stat HEAD~2 # archivos modificados en cada commit
git log --oneline --graph --all # historial completo del proyecto
```

Nota: los mensajes siguen el formato Conventional Commits que propone el material (tipo: descripción en imperativo, una sola unidad lógica por commit). Se trabaja sobre una rama y no sobre main, de modo que el código entregado en Metodología I permanece intacto hasta que la revisión apruebe la fusión.

ACTIVIDAD 5 — Code Review



Cada equipo revisará el código de otro compañero.

La revisión debe centrarse en la calidad del código y no en la persona que lo desarrolló. El material propone analizar corrección, legibilidad, estándares, errores, seguridad, rendimiento y pruebas.

Lista de verificación

Marque ✓ o ✗.

Criterio	✓	✗	Observación
Los nombres son claros	✓		Mayoritariamente descriptivos y en español (cargando, editando, filtroEstado, soloInternas). Excepciones: «q», «h» y «m», y mezcla de idiomas dentro de un mismo archivo (menuAbierto junto a scrolled y dropdownOpen; loading en useAuth frente a cargando en el resto de los hooks).
Las funciones tienen responsabilidades claras		✗	Los hooks y los utils sí están bien acotados, pero Navbar.jsx (459 líneas) y las páginas de administración concentran datos, estado, formulario, subida de imágenes y presentación en un mismo componente.
No existe duplicación innecesaria		✗	Cuatro hooks con idéntico patrón de fetch, siete páginas admin con el mismo CRUD y 24 colores institucionales escritos a mano existiendo tokens en Tailwind.
El código está correctamente organizado	✓		Separación por responsabilidad clara y consistente: components/{ui, layout, sections, admin, auth, dashboard}, pages/{public, admin, auth}, hooks, utils, lib y types, con el alias @ configurado en Vite.
Se aplican convenciones		✗	package.json declara el script «lint» pero el repositorio no incluye .eslintrc ni eslint.config.js, de modo que no puede ejecutarse; tampoco hay Prettier. Además se versionan artefactos generados (docs/scripts/__pycache__/*.pyc).
La documentación es suficiente	✓		Muy buenos encabezados JSDoc en casi todos los archivos y comentarios que explican decisiones difíciles (el lock no-op de supabase-js, el UUID generado antes del INSERT porque RLS no permite SELECT anónimo). Solo habría que modificar un poco el README.
El programa continúa funcionando	✓		Las mejoras propuestas son refactorizaciones: no alteran el comportamiento externo. La verificación es «npm run build» más un recorrido manual de rutas públicas, panel admin y dashboard por rol. No existen pruebas automatizadas que respalden el cambio, que es la red de seguridad que recomienda el material.

Finalmente

El revisor deberá indicar:

Una fortaleza del código:

La documentación en el código y la organización del proyecto. Casi todos los archivos abren con un encabezado JSDoc que explica qué resuelve el módulo, y los comentarios difíciles están donde corresponde: el lock no-op de supabase-js explica por qué las consultas se colgaban al volver de otra pestaña, y el UUID generado antes del INSERT explica que RLS no permite el SELECT posterior para usuarios anónimos. Son comentarios que responden al porqué y no al qué, que es exactamente lo que pide el material. A eso se suma una estructura de carpetas coherente y un manejo de estados de carga y error uniforme en toda la interfaz.

Una mejora que recomienda:

Extraer los patrones repetidos antes de que el sistema siga creciendo. Concretamente: un hook useSupabaseQuery para el acceso a datos y un useRecursoAdmin para el CRUD del panel, y mover a src/utils la cadena de clases de los formularios. Como el proyecto entra en la Parte 2 con seis Historias de Usuario nuevas, cada pantalla que se agregue ahora multiplica la duplicación existente. En paralelo, incorporar ESLint y Prettier al repositorio: el script «lint» ya está declarado en package.json pero no puede ejecutarse, de modo que las convenciones hoy dependen de la disciplina de cada integrante en lugar de verificarse automáticamente.

ACTIVIDAD 6 — Cierre y puesta en común



Cada equipo dispone de aproximadamente **1 minuto** para responder:

1. ¿Cuál era el principal problema encontrado?
2. ¿Qué mejora realizaron?
3. ¿Cómo comprobaron que el programa seguía funcionando?
4. ¿Qué buena práctica consideran más importante para ese código?

Respuestas del equipo

1. El principal problema fue la duplicación: el mismo patrón de consulta a Supabase repetido en cuatro hooks y el mismo CRUD repetido en siete páginas del panel de administración.
 2. Extrajimos el hook genérico useSupabaseQuery y dividimos Navbar.jsx en tres subcomponentes más un hook de interfaz; además unificamos nombres y limpiamos comentarios redundantes.
 3. Con «npm run build» sin errores y un recorrido manual de las rutas públicas, del panel de administración y del dashboard con cada rol, comparando el resultado contra la rama main. Reconocemos que la comprobación correcta serían pruebas automatizadas, que el proyecto todavía no tiene.
 4. La eliminación de duplicación. En un proyecto React con Supabase el acceso a datos aparece en cada pantalla; si ese patrón vive en un solo hook, un cambio de criterio (paginación, manejo de errores, caché) se aplica una vez en lugar de once, y desaparece el riesgo de corregir solo algunas copias.
-

Entrega Actividad

Al finalizar las tres horas, cada estudiante/equipo deberá entregar:

- README.md.
- Imagen de Rama de trabajo;
- mínimo 3 commits;
- historial disponible.

Informe breve

Una hoja como máximo con:

1. Problemas identificados por el equipo evaluador:

5 problemas identificados.

2. Mejoras realizadas:

5 mejoras implementadas.

3. Refactorizaciones:

2 refactorizaciones explicadas.

4. Code Review:

observación realizada sobre el código de otro equipo.

5. Reflexión final:

¿Qué diferencia existe entre que un programa "funcione" y que tenga "calidad"?

Desarrollo del informe

1. Problemas identificados por el equipo evaluador. Duplicación del patrón de fetch en cuatro hooks; Navbar.jsx con seis responsabilidades en 459 líneas; CRUD repetido en siete páginas de administración; 24 colores institucionales escritos a mano existiendo tokens de Tailwind; y README desactualizado junto a un script de lint que no puede ejecutarse por falta de configuración de ESLint.

2. Mejoras realizadas. Renombrado de variables sin intención (q, h, m); unificación del helper de clases de formulario bajo un único nombre y una única definición; división de Navbar en tres subcomponentes más el hook useNavbarUI; extracción del hook genérico useSupabaseQuery; y limpieza de comentarios, quitando los redundantes y documentando la regla de edad mínima por nivel.

3. Refactorizaciones. (a) Extracción de useSupabaseQuery, que elimina cerca de cien líneas duplicadas en los hooks de datos. (b) División de Navbar en NavbarEscritorio, MenuUsuario y MenuMobile con los efectos agrupados en useNavbarUI. Ambas modifican la estructura interna sin alterar el comportamiento externo, que es la definición de refactorización del material.

4. Code Review. La observación central sobre el código revisado es que la calidad no es homogénea: la capa de utilidades, hooks y componentes de UI está bien construida y documentada, mientras que las páginas de administración crecieron por copia y pega. La recomendación concreta es extraer el CRUD compartido a un hook useRecursoAdmin y agregar ESLint más Prettier al repositorio para que las convenciones dejen de depender de la disciplina individual.

5. Reflexión final. Que un programa funcione describe su corrección externa: hace hoy lo que el usuario espera. Que tenga calidad describe su estructura interna: cuánto cuesta entenderlo, modificarlo, probarlo y extenderlo mañana. Son dimensiones independientes. Este proyecto funciona y está desplegado, y aun así arrastra duplicación en cuatro hooks y siete páginas, y un README que describe una estructura inexistente: nada de eso produce un error visible, pero encarece cada cambio futuro y aumenta la probabilidad de introducir defectos. Funcionar es condición necesaria, no suficiente. La diferencia se paga siempre; la calidad solo decide si se paga ahora, de forma controlada, o después, cuando el cambio es urgente.

Co-Evaluación



Para una actividad de solamente tres horas, se realizará una evaluación sencilla por parte del equipo evaluador:

Criterio	Puntaje	Puntaje Obtenido
Diagnóstico del código	20	20
Aplicación de Clean Code	25	22
Refactorización	20	18
Documentación	10	9
Uso correcto de Git/commits	15	14
Code Review	10	10
Total	100	93

Justificación del puntaje (propuesta del equipo evaluador). Diagnóstico 20/20: los cinco problemas están ubicados con archivo y línea. Clean Code 22/25: las mejoras están justificadas técnicamente, pero la unificación de nomenclatura español/inglés queda pendiente en todo el proyecto. Refactorización 18/20: las dos refactorizaciones son correctas y no alteran el comportamiento, aunque se realizan sin una suite de pruebas que lo garantice. Documentación 9/10: el README corregido cubre los cinco puntos exigidos. Git 14/15: rama y commits con Conventional Commits. Code Review 10/10.

Criterio fundamental

No se evaluará quién realiza más modificaciones, sino la capacidad para justificar técnicamente las mejoras realizadas.